Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Cloud-Native Automated Deployment
Class Applied Computing Name Zhou Jiaqi ID 202483910036

1. Experimental Purpose

The primary goal of this experiment is to introduce students to the core concepts of Cloud Computing through hands-on practice with Platform-as-a-Service (PaaS) and Automation. By completing this lab, students will gain practical experience with GitHub-based cloud deployment pipelines.

2. Experiment Contents

This lab consists of five integrated modules designed to simulate a professional cloud deployment lifecycle:

• Resource Acquisition (Forking repository)

• Enabling the Hosting Service (GitHub Pages)

• Triggering Cloud Automation (GitHub Actions)

• Verification & Global Access

• Content Personalization & Elements (optional)

3. Detailed Experimental Procedure

3.1 Step 1: Resource Acquisition

Log in to GitHub and navigate to the repository: https://github.com/yhldrf/cloud-computing-lab-github. Click the Fork button to create a private instance of the project in the cloud ecosystem.

Cloud Concept: Forking creates a personal copy of the repository in the cloud, enabling independent development and experimentation.

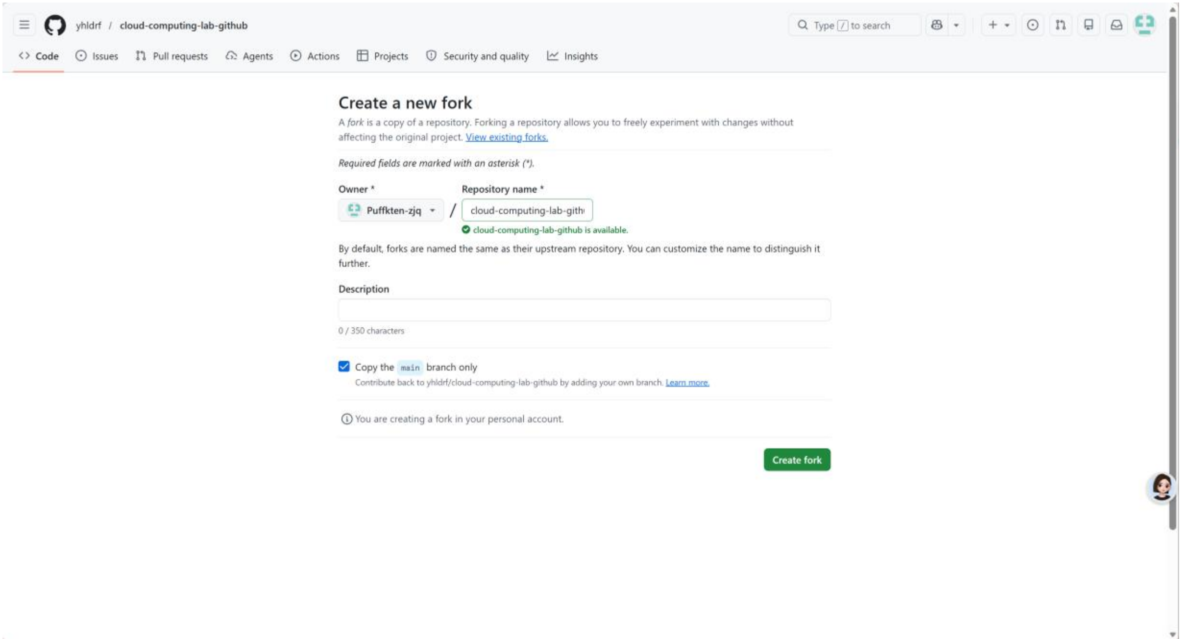

Figure 1: Forking the GitHub repository

3.2 Step 2: Enabling the Hosting Service

Go to Settings > Pages. Under Build and deployment > Source, select "GitHub Actions". This switches from manual upload to a managed pipeline service.

Cloud Concept: This demonstrates the shift from manual infrastructure management to automated, managed services - a key characteristic of cloud computing.

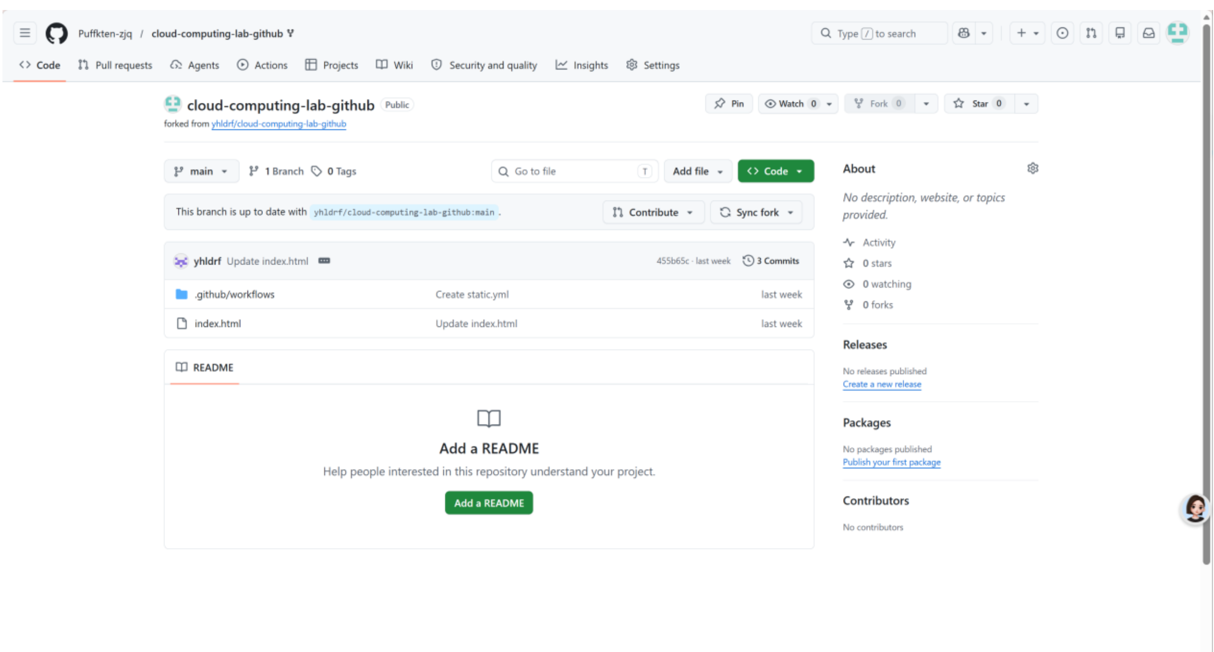

Figure 2: Configuring GitHub Pages settings

3.3 Step 3: Triggering Cloud Automation

Open index.html in the browser editor. Change [Enter Your Name Here] to your actual name. Click Commit Changes to trigger the automated deployment pipeline.

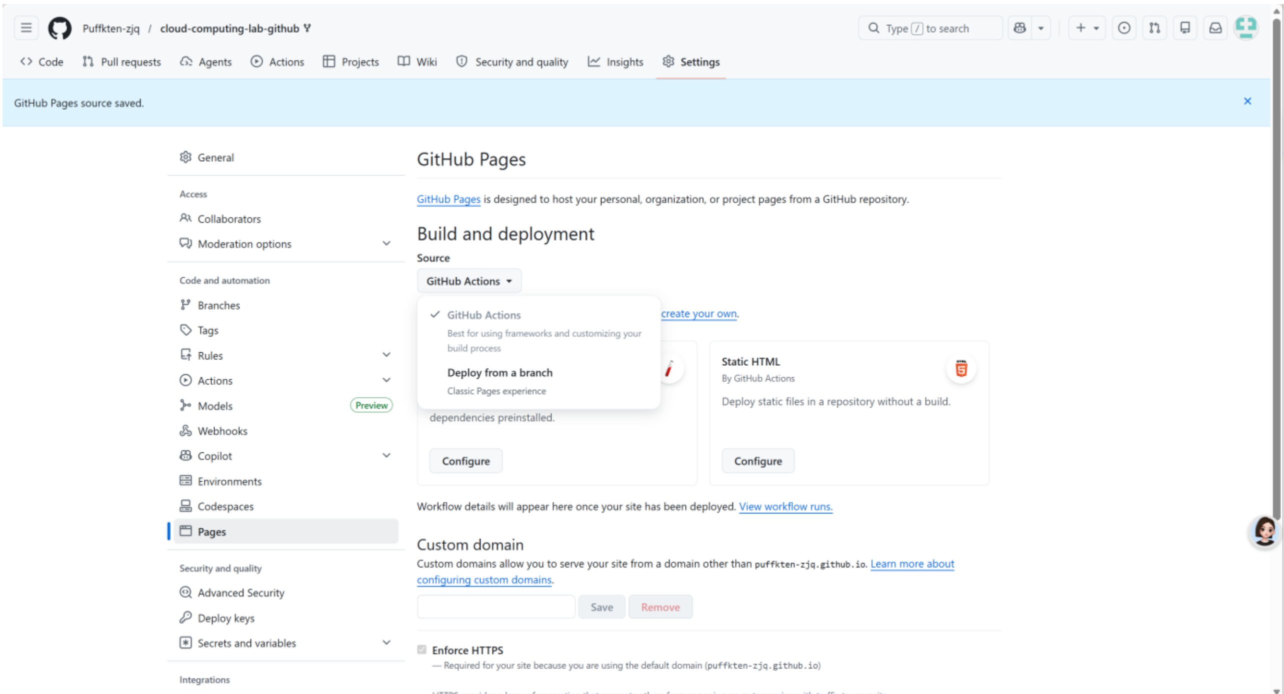

Figure 3: Editing index.html in GitHub editor

Immediately click the Actions tab at the top of the page to observe the Cloud-Deployment-Pipeline start.

Cloud Concept: GitHub spins up a Virtual Machine (Ubuntu) to process the request. This exemplifies "On-demand Computing" where resources are allocated dynamically.

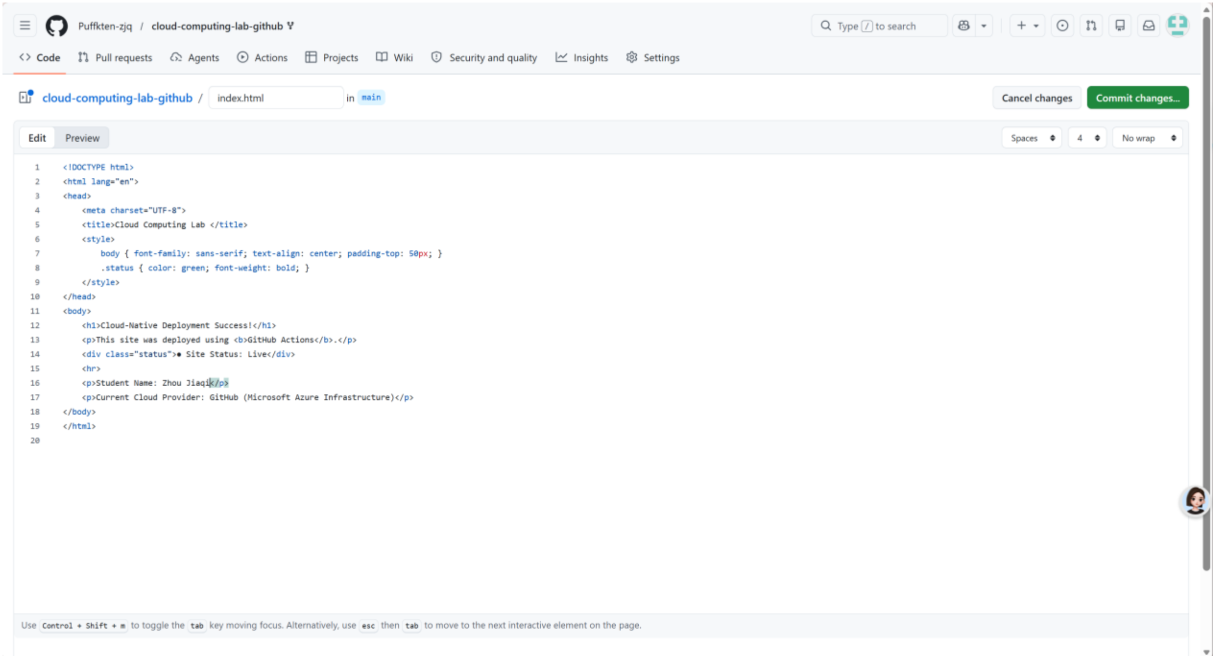

Figure 4: GitHub Actions workflow in progress

3.4 Step 4: Verification & Global Access

Once the Action finishes (indicated by a green checkmark), go back to Settings > Pages to find the Live URL of the deployed website.

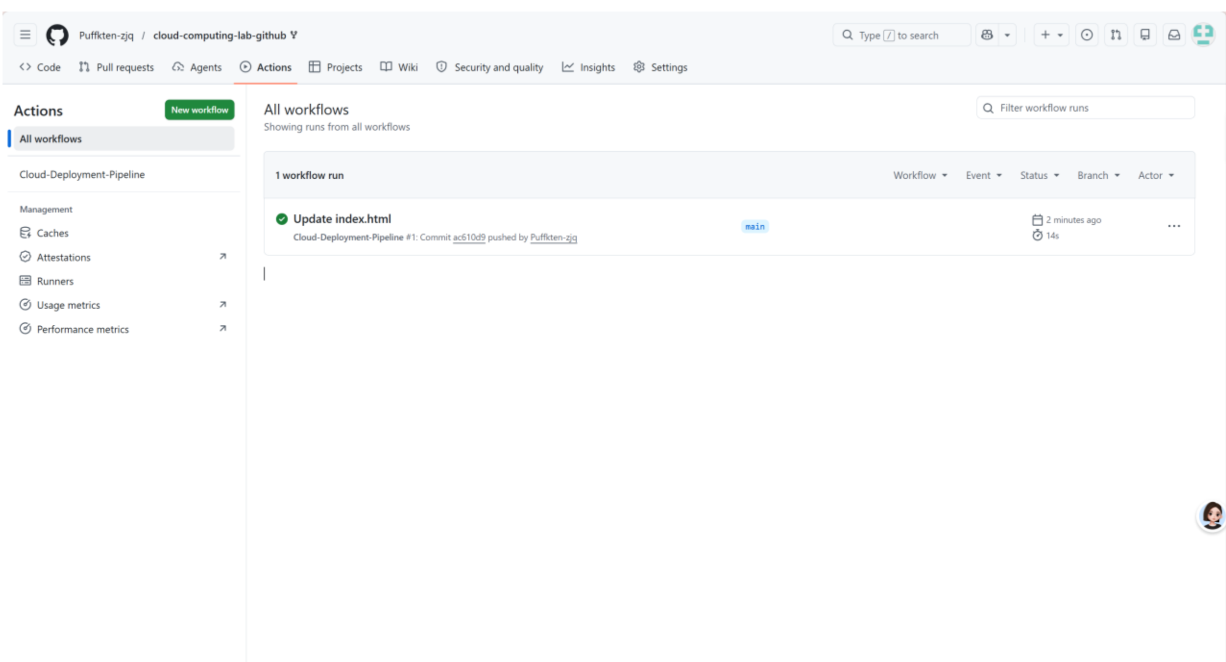

Figure 5: Actions workflow completed successfully

Open the URL in a browser to verify the deployment.

Cloud Concept: The site is deployed via PaaS (Platform as a Service) model. The cloud provider manages the underlying infrastructure and global delivery.

Figure 6: Successfully deployed website

3.5 Step 5: Content Personalization & Elements (Optional)

Beyond changing the name, additional customizations were made to demonstrate how the cloud updates different types of content:

• Theme Color Modification (CSS Styles):

Modified the <style> block to change background color and font settings. This demonstrates how UI configurations are stored and managed in the cloud.

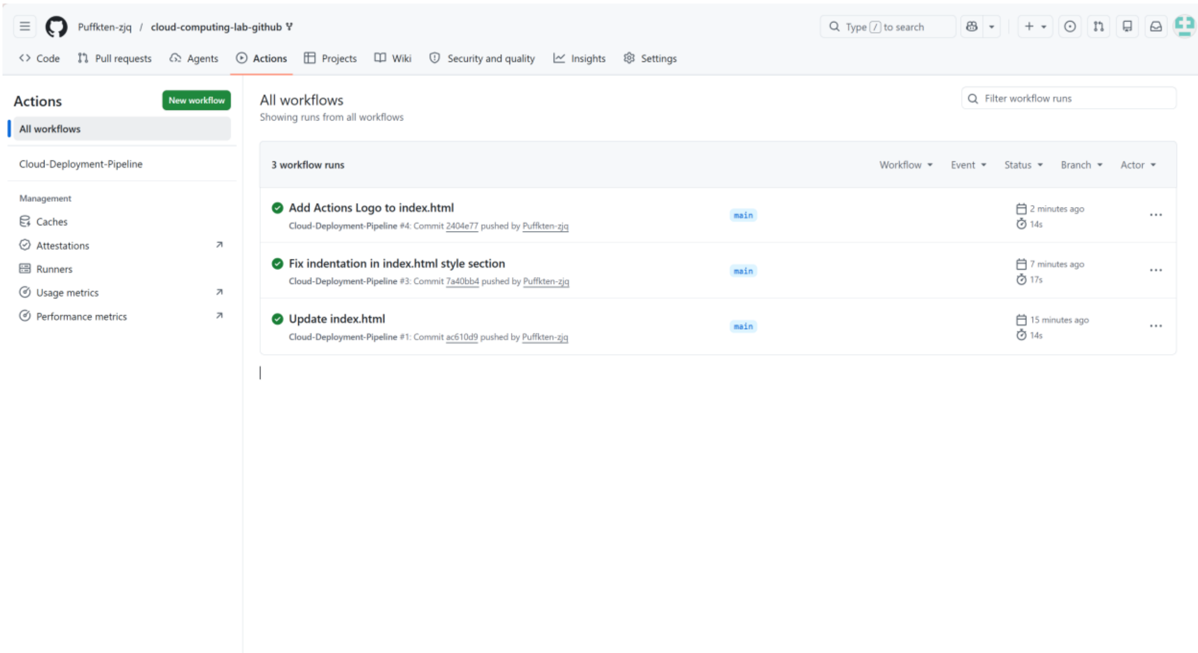

Figure 7: CSS style modifications

• Multimedia Integration (Images):

Added an <img> tag to demonstrate how the cloud handles external resources. This showcases the flexibility of cloud-based content delivery.

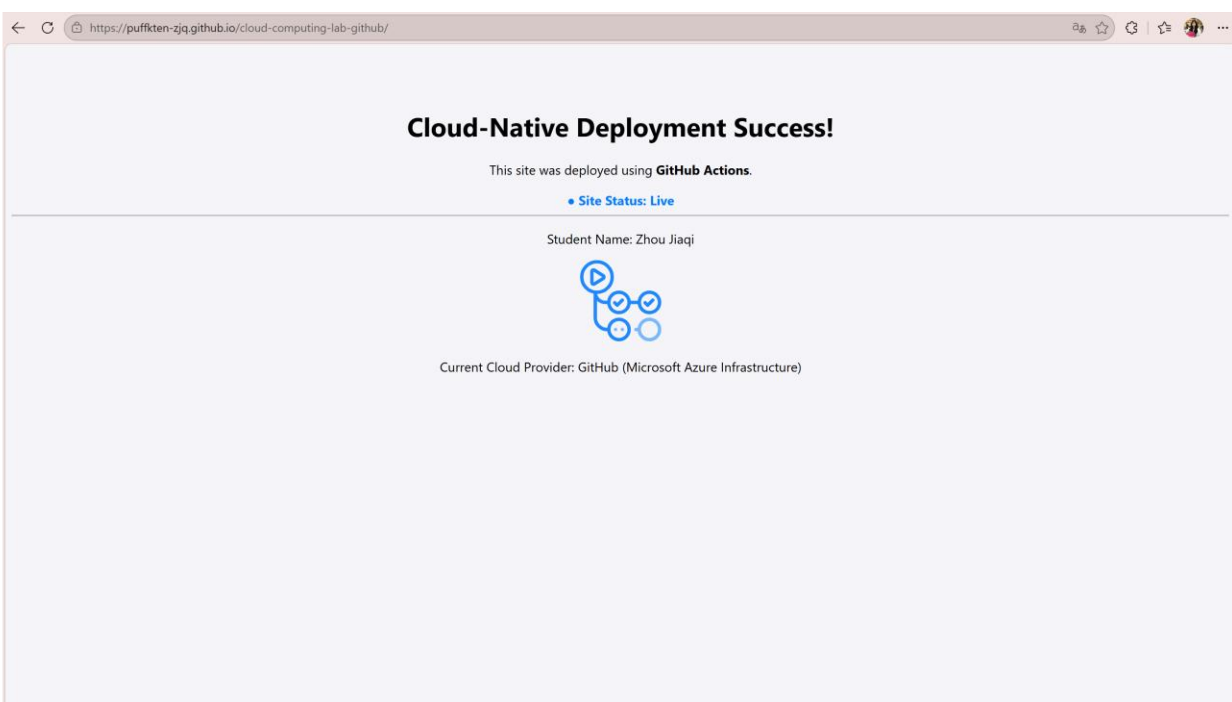

Figure 8: Multimedia integration in the deployed site

4. Reflection

Q1: Where is your website stored?

The website is stored on GitHub's cloud infrastructure. GitHub Pages uses a global CDN (Content Delivery Network) to host and distribute the content, ensuring fast access from anywhere.

Q2: Did you have to install any software?

No software installation was required. All operations were performed through a web browser, demonstrating the "Everything as a Service" (XaaS) paradigm where tools and infrastructure are delivered over the internet.

5. Problems Encountered and Proposed Solutions

During the experiment, several issues were encountered and resolved:

Issue 1: Actions workflow failed initiall

Solution: Verified that the Pages source was correctly set to "GitHub Actions" and ensured the workflow file existed in the repository.

Issue 2: Deployment URL not appearing immediately

Solution: Waited for the Actions workflow to complete and refreshed the Settings > Pages page. The URL appeared once the deployment finished.

Issue 3: Image rendering issues

Solution: Ensured the image URL was accessible and properly formatted. Verified the image tag syntax was correct before committing changes.

6. Conclusion

This experiment successfully demonstrated the core concepts of cloud-native automated deployment. Through hands-on practice with GitHub Pages and Actions, students gained practical experience with PaaS, CI/CD pipelines, and on-demand computing. The experiment effectively illustrates how cloud computing enables rapid, scalable deployment without requiring local infrastructure.